O-NOTATION

Beschreibt, wie der Aufwand wächst, wenn die Datenmenge n wächst. Konstante Faktoren fallen weg. **log n:** Wie oft kann man n halbieren, bis 1 übrig ist? (n = 1 000 → etwa 10)

Klasse	Name	Beispiel	n = 1 000	n = 1 000 000
O(1)	konstant	Zugriff auf a[5]	1	1
O(log n)	logarithmisch	binäre Suche	≈ 10	≈ 20
O(n)	linear	lineare Suche	1 000	1 000 000
O(n log n)	n log n	Merge Sort	≈ 10 000	≈ 20 000 000
O(n²)	quadratisch	Bubble Sort	1 000 000	10^{12}

- Schnell → langsam: O(1) < O(log n) < O(n) < O(n log n) < O(n²).
- n verdoppelt: O(1) gleich, O(log n) +1 Schritt, O(n) ×2, O(n²) ×4.

LINEARE SUCHE

- Alle Elemente der Reihe nach mit dem gesuchten Wert vergleichen.
- **Voraussetzung:** keine (auch unsortiert, auch Listen). **Aufwand:** O(n), schlechtester Fall n Vergleiche.

BINÄRE SUCHE

Voraussetzung: sortiert + Indexzugriff (Array). **Aufwand:** O(log n).

1. lo = erster Index, hi = letzter Index.
2. m = (lo + hi) / 2, abgerundet.
3. a[m] = gesucht → fertig.
4. Gesucht größer → lo = m + 1; kleiner → hi = m − 1.
5. Wiederholen. lo > hi → nicht vorhanden.

Suche 61 in [10, 19, 23, 25, 36, 61, 79, 81, 99] (Index 0–8):

Schritt	Bereich	m	a[m]	Entscheidung
1	0–8	4	36	61 > 36 → rechts
2	5–8	6	79	61 < 79 → links
3	5–5	5	61	gefunden

Höchstzahl Schritte: kleinstes k mit $2^k > n$.

n	15	100	500	1 000	1 000 000
k	4 ($2^4 = 16$)	7 (128)	9 (512)	10 (1 024)	20 (≈ 1,05 Mio.)

BUBBLE SORT

1. Von vorne je zwei Nachbarn vergleichen.
2. Falsch herum → tauschen.
3. Bis zum Ende des unsortierten Teils; das letzte Element ist fertig.
4. Neuer Durchlauf; Ende, wenn ein Durchlauf ohne Tausch bleibt.

[5, 1, 4, 2, 8]:

- Durchlauf 1: 5↔1, 5↔4, 5↔2 → [1, 4, 2, 5, 8] (8 fertig)
- Durchlauf 2: 4↔2 → [1, 2, 4, 5, 8]
- Durchlauf 3: kein Tausch → sortiert

Nach Durchlauf 1 steht das **größte** Element hinten. Höchstens n − 1 Durchläufe. **Voraussetzung:** keine. **Aufwand:** O(n²).

MERGE SORT

Teile und Herrsche, rekursiv.

1. Array in der Mitte teilen.
2. Weiter teilen, bis jedes Stück 1 Element hat (immer sortiert).
3. Je zwei sortierte Stücke mergen: vorderste Elemente vergleichen, kleineres übernehmen.
4. Bis nur ein Stück übrig ist.

```
Teilen:   [38, 27, 43]          [3, 9, 82, 10]
          [38] [27, 43]         [3, 9] [82, 10]
          [38] [27] [43]        [3] [9] [82] [10]
Mischen:  [27, 38, 43]          [3, 9, 10, 82]
          [3, 9, 10, 27, 38, 43, 82]
```

Letzter Merge: 27/3 → 3, 27/9 → 9, 27/10 → 10, 27/82 → 27, 38/82 → 38, 43/82 → 43, Rest 82 anhängen.

Voraussetzung: keine. **Aufwand:** O(n log n): ≈ $\log_2$ n Ebenen, je Ebene jedes Element einmal.

VERGLEICH

Verfahren	Voraussetzung	Prinzip	Aufwand
lineare Suche	keine	alle nacheinander	O(n)
binäre Suche	sortiert, Index	Mitte, Hälfte verwerfen	O(log n)
Suche im Binärbaum	geordneter Binärbaum	links kleiner, rechts größer	balanciert O n), degenerie O(n)
Bubble Sort	keine	Nachbarn tauschen	O(n²)
Merge Sort	keine	teilen, mischen	O(n log n)
Quicksort	keine	Pivot, kleiner/größer trennen, rekursiv	Mittel O(n lo schlimmster O(n²)
Selection Sort	keine	kleinstes suchen, nach vorne tauschen	O(n²)

TYPISCHE FEHLER

- Binäre Suche auf unsortierten Daten.
- Binäre Suche als O(n log n) angeben. Richtig: O(log n); O(n log n) ist Merge Sort.
- Bei gerader Anzahl die Mitte mal ab-, mal aufrunden. Bei einer Regel bleiben.
- „Nach Durchlauf 1 steht das kleinste vorne." Das größte steht hinten.
- Bubble Sort als O(n log n) angeben.
- Bei Merge Sort beim Teilen sortieren. Geteilt wird stumpf in der Mitte, sortiert beim Mischen.
- Lineare Suche als „schlecht" abtun: bei unsortierten Daten die einzige Möglichkeit.
- Matrixmultiplikation (n × n) unter O(n²) einordnen, wie auf der Folie. Das einfache Verfahren ist O(n³).